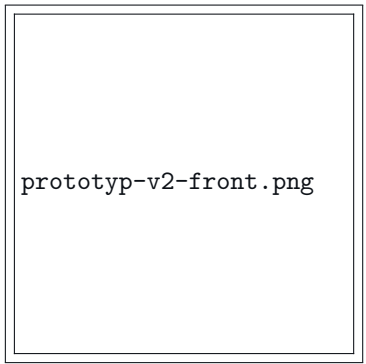
Prototyp: 10x Taster/LED · FreeRTOS + React · WebSocket

Jan Unger

Stand: 17.01.2026 | Prototyp 10x **funktionsfaehig** | Vollausbau 100x **in Planung**

Hardware-Prototyp v2



Bestückungsseite – ESP32 + ICs



KiCad – Schaltplan

Komponenten: ESP32-S3 XIAO • 2× 74HC595 • 2× CD4021BE • 10 LEDs • 10 Taster

Schieberegister-Architektur



Komponenten

- ESP32-S3 XIAO – Mikrocontroller
- 2× 74HC595 – LED-Ausgabe (Active-High)
- 2× CD4021BE – Taster-Eingabe (Active-Low)

Signale (SPI-ähnlich)

- Data: MOSI → 74HC595 SER, MISO ← CD4021 Q8
- Clock: SCK gemeinsam (SRCLK/CLK)
- Strobes: RCK (595 Latch), P/S (4021 Load/Shift)

Prototyp-Evolution

[v1-front.png]

v1 Bestückung – Sockel ohne ICs

[v1-back.png]

v1 Lötseite

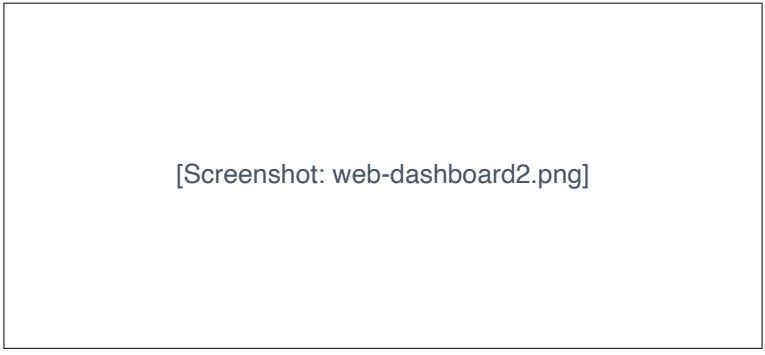
prototyp-v2-front.png

v2 Bestückung

[v2-back.png]

v2 Lötseite

Web-Dashboard



Zugriff
`rover.local:8081`

Multi-Device
iMac, iPad, iPhone

React + Vite
TypeScript, HMR

Synchron
WebSocket Broadcast

Was ist das Selection Panel?

Anwendungsfall

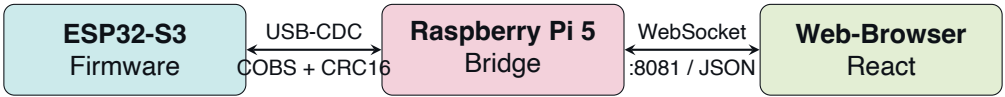
- Prototyp: 10 physische Taster
- Vollausbau: 100 Taster geplant
- LED zeigt aktive Auswahl an
- Web-Dashboard visualisiert Status

Policy: „LowestIdWins“

- Neuer Tastendruck $\rightarrow$ **sofortiger Wechsel**
- Bei Mehrfachdruck: niedrigste ID gewinnt
- **One-hot LED:** Immer max. 1 LED aktiv
- Selection $= 0 \rightarrow$ alle LEDs aus

Merksatz: Bei gleichzeitigem Druck gewinnt die niedrigste ID – deterministisch und vorhersagbar.

Systemarchitektur – Überblick



Komponente	Aufgabe
ESP32-S3 XIAO	Taster scannen, LEDs steuern (10× Prototyp / 100× Vollausbau)
Raspberry Pi 5	Protokoll-Bridge (`bridge.py`), WebSocket-Server, Static-Files
Browser (React)	Auswahl visualisieren, WebSocket-Client

Hardware – Schieberegister

Warum Schieberegister?

- Prototyp: 20 IOs mit **6 GPIO-Pins**
- Vollausbau: $13 \times 8 = 104$ Kanäle (100 genutzt)
- Kaskadierbar, günstig und robust

Pinbelegung ESP32-S3 (gemeinsamer SPI)

Signal	Pin	Funktion
SCK	D8	SPI Clock (gemeinsam)
MOSI	D10	74HC595 SER
MISO	D9	CD4021BE Q8
RCK	D0	74HC595 Latch
OE	D2	74HC595 Enable (Active-Low)
P/S	D1	CD4021BE Parallel/Shift



CD4021BE Timing-Diagramm

Firmware – Modulare FreeRTOS-Architektur

IO-Task (Core 0, Prio 1)

- Polling alle 10 ms (100 Hz)
- Debouncing pro Taste (50 ms)
- Selection via IoPipeline (LowestIdWins)

Serial-Task (Core 1, Prio 1)

- COBS-Framing (0x00 Delimiter)
- CRC-16 CCITT-FALSE
- ACK für Host-Kommandos (optional via Flag)

Schichtenmodell

Schicht	Inhalt
app/	io_task, serial_task
logic/	debounce, selection, io_pipeline
drivers/	cd4021, hc595
hal/	spi_bus + Mutex

SPI-Konfiguration

- CD4021BE: MODE1, 500 kHz
- 74HC595: MODE0, 1 MHz
- Mutex für Thread-Safety

Serial-Protokoll

Frame-Aufbau (COBS-kodiert)

Feld	Größe
Magic ('SP')	2 Bytes
Version	2 Bytes
Type	1 Byte
Flags	1 Byte
Sequence	2 Bytes (LE)
Length	2 Bytes (LE)
Payload	0–256 Bytes
CRC-16	2 Bytes (LE)

Header: 10 Bytes + Payload + 2 Bytes CRC

Nachrichtentypen (Firmware)

- `0x01 HELLO` – Konfiguration (10 Bytes)
- `0x12 SELECTION_STATE` – Kanalwechsel (8 Bytes)
- `0x20 CMD_SET_SELECTION` – Host-Befehl (4 Bytes)
- `0x30 ACK` – Bestätigung (0 Bytes)

ACK/Retry (Host → ESP32)

- Jede TX bekommt eine `seq` (1..65535, 0 wird übersprungen)
- Parameter: `--ack-timeout` (typ. 0.6 s), `--retries` (typ. 4)
- Bei Timeout: `warn` (code=20), bei Fail: `error` (code=21)

Raspberry Pi Bridge

Technologie-Stack:

- **aiohttp:** Async HTTP/WebSocket-Server
- **pyserial-asyncio:** Async Serial I/O
- **COBS/CRC:** Frame-Kodierung/-Prüfung
- **systemd:** Autostart als Service (optional mit `flock`)

Datenfluss bei Tastendruck:

1. ESP32 sendet `SELECTION_STATE` (z. B. `selected=3`, `source=local_btn`)
2. Bridge speichert State, broadcastet JSON an alle Browser
3. Browser empfängt `{"type":"selection","selected":3,"source":"local_btn"}`

Datenfluss bei Browser-Klick:

1. Browser sendet `{"type":"setSelection","selected":5}`
2. Bridge sendet `CMD_SET_SELECTION` mit `seq` an ESP32
3. ESP32 antwortet (optional) mit `ACK`, danach `SELECTION_STATE`
4. Bridge broadcastet neuen State an alle Browser

Dashboard – Implementierung

Features

- **React + Vite:** HMR, TypeScript
- **WebSocket:** Auto-Reconnect (z. B. 1 s)
- **Build (PROD):** Output nach `dashboard-prod-pi/static/`
- **Dev-Proxy:** zu `rover.local:8081`

Farbschema (10 Kanäle)

- Weiß (1–2), Blau (3–4)
- Rot (5–6), Gelb (7–8)
- Grün (9–10), zyklisch erweiterbar

Statusleiste-Anzeige

Feld	Bedeutung
IN	Eingangsbits (Active-Low)
OUT	Ausgangsbits (Active-High)
sel	Aktuell gewählter Kanal
src	Quelle (local_btn / host_cmd)
tMs	Timestamp (ms seit Boot)

Bedienung

- Klick auf Kanal-Tile → `setSelection`
- „Alle aus“ → `selected=0`

Getestete Funktionen

Firmware (ESP32)

- Modulare Architektur (HAL/Drivers/Logic/App)
- SPI-Bus mit Mutex (Thread-safe)
- Debouncing: 50 ms stabil
- Selection: One-hot LED-Ansteuerung (LowestIdWins)
- COBS/CRC16 Frame-Handling

Bridge (Raspberry Pi)

- Async Serial (pyserial-asyncio)
- ACK/Retry für Host-Kommandos (parametrierbar)
- WebSocket Broadcast funktioniert
- State-Recovery bei Reconnect

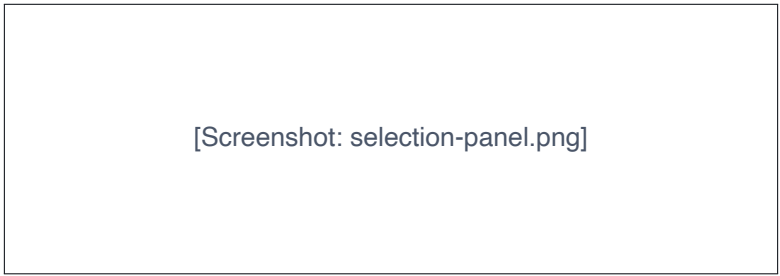
Dashboard (React)

- 10-Kanal-Visualisierung
- WebSocket-Verbindung stabil
- Statusleiste zeigt Live-Daten
- Build-Pipeline (Vite $\rightarrow$ `static/`)

Hardware-Prototyp

- 10 Taster funktionsfähig
- 10 LEDs funktionsfähig
- Bit-Mapping korrekt (MSB-first)
- CD4021BE + 74HC595 stabil

Demo (End-to-End)



1. Button k wird gedrückt $\rightarrow$ ESP32 setzt `selected=k` (LowestIdWins).
2. ESP32 sendet `SELECTION_STATE` über USB-CDC (COBS + CRC16).
3. Bridge broadcastet JSON über `/ws` an alle Browser.

Live-Demo

Dashboard öffnen (alle Geräte):

```
http://rover.local:8081/
```

WebSocket-Test mit wscat:

```
wscat -c ws://rover.local:8081/ws
> {"type":"setSelection","selected":5}
```

Service-Status prüfen:

```
sudo systemctl status embedded-bridge.service
```

Serial-Port prüfen:

```
ls -la /dev/serial/by-id/ | grep Espressif
```

Hardware: Taster drücken → LED leuchtet, Status auf allen Browsern synchron.

Ausblick: Vollausbau 100×

[Blockdiagramm: vollausbau-100x.png]

Änderungen für 100×

- 13× 74HC595 (statt 2×)
- 13× CD4021BE (statt 2×)
- `BTN_COUNT = 100`
- `LED_COUNT = 100`
- Gleiche Firmware/Bridge (nur Konfiguration)

Stückliste (100×)

Bauteil	Anzahl
74HC595	13×
CD4021BE	13×
LED 5 mm	100×
Taster	100×
R 330 Ω – 3 k (LED)	100×
R 10 k (Pull-up)	100×
C 100 nF	26×

Zusammenfassung

Phase	Inhalt	Status
1	Infrastruktur & Toolchain	✓
2	Hardware-Prototyp (10×)	✓
3	ESP32 Firmware (FreeRTOS)	✓
4	Serial-Protokoll (COBS + CRC16)	✓
5	Raspberry Pi Bridge	✓
6	React-Frontend	✓
7	Hardware-Integration (10×)	✓
8	Hardware-Skalierung (100×)	◀ Nächste Phase

Kernbotschaft: ESP32 für Echtzeit-I/O, Pi als Protokoll-Bridge, Browser für UI. Skalierung 10 → 100 durch Konfiguration.

Übertragbarkeit: Voting-Systeme, Quiz-Panels, Museumsinstallationen.

Credits

KI-Unterstützung	**Claude Opus 4.5** • **ChatGPT 5.2** Text- und Code-Vorschläge
Qualitätssicherung	Technische Prüfung durch den Autor